A Survey on Information Security Using Artificial Neural Network

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELOR OF SCIENCE (HONOURS)
IN

COMPUTER SCIENCE

Submitted by

Arka Maity

Debjit Dey

Shouvik Sarkar

EXAMINATION ROLL NO.:

2024161202, 2024161212, 2024161208

REGISTRATION:

A01-1112-117-008-2021, A01-1112-117-020-2021, A01-1112-117-026-2021

PAPER CODE: CMSA DSE-VI

SEMESTER: VI

Under the supervision of

Dr. Soumya Paul



DEPARTMENT OF COMPUTER SCIENCE
RAMAKRISHNA MISSION VIVEKANANDA CENTENARY COLLEGE
Rahara, Khardaha, West Bengal 700118

April, 2024

DECLARATION

We, Arka Maity, Debjit Dey & Shouvik Sarkar, Examination Roll No.: 2024161202, 2024161212, 2024161208, Registration No.: A01-1112-117-008-2021, A01-1112-117-020-2021, A01-1112-117-026-2021 respectively, Students of B.Sc. (Hons.) in Computer Science, hereby declare that this project, report titled **"A Survey on Information Security Using Artificial Neural Network"** which is submitted by us to the Department of Computer Science, Ramakrishna Mission Vivekananda Centenary College, Rahara, Kolkata in partial fulfilment of the requirement for the award of degree of Bachelor of Science (HONOURS) is a presentation of our own original work and not copied from any other sources without proper citation and has not been previously included in a project report/dissertation/thesis submitted to this or any other institution for a degree, diploma or any other qualification. The project work was done under the supervision of Dr. Soumya Paul.

Place: **Kolkata**
Date: **20/04/2024**

Arka Maity **Debjit Dey** **Shouvik Sarkar**

CERTIFICATE

This is to certify that the project work entitled “A Survey on Information Security using Artificial Neural Networks” submitted by Arka Maity, Debjit Dey and Shouvik Sarkar, in partial fulfillment of the requirements for the BSc. (H) in Computer Science under my guidance and supervision, is an acceptable work.

I wish every success in life.

Prof. (Dr.) Soumya Paul
Principal and Professor

Dinabandhu Andrews Institute of Technology and Management.

Acknowledgement

We wish to express our sincerest gratitude to Prof. Dr. Soumya Paul for the continuous guidance and mentorship that he provided us during the project. He showed us the path to achieve our targets by explaining all the tasks to be done and explained to us the importance of this project as well as its industrial relevance. He was always ready to help us and clear our doubts regarding any hurdles in this project. Without his constant support and motivation, this project would not have been successful.

Our Sincere thanks and gratitude to our revered Principal Sw. Kamalasthananda Ji Maharaj and revered Controller of Examination Sw. Vedanuragananda -Ji Maharaj for allowing us to explore this domain of our subject and for constant support throughout the project. We are thankful to all faculty and staff members of the Dept. of Computer Science for their guidance and support.

I would like to say thanks to all my friends for their continuous support and motivation.

Last but not the least, I am thankful to my parents for their continuous support which helped me to work hard to complete this project.

Place: Kolkata
Date: 20/04/2024

Arka Maity **Debjit Dey** **Shouvik Sarkar**

Abstract

Ensuring information security in long-distance transmissions across computer networks presents significant challenges. This survey explores the application of artificial neural networks (ANNs) in addressing these challenges through the development of a novel symmetric key cryptographic system. Leveraging ANNs, the system aims to enhance data confidentiality and message authentication during transmission.

The proposed system focuses on three key objectives: enhanced data confidentiality, robust message authentication, and long-distance network resilience. To achieve enhanced data confidentiality, the system utilizes ANNs to create a dynamic and unpredictable encryption process, significantly increasing the difficulty of unauthorized decryption. Robust message authentication is ensured by incorporating ANNs to verify the message's origin and prevent tampering, thereby maintaining data integrity and thwarting forgery attempts.

Moreover, the system is designed with long-distance network resilience in mind, considering factors such as potential network delays and varying data packet sizes. By leveraging ANNs, the system adapts to these challenges, ensuring effective operation across diverse computer networks.

Through this innovative approach, the proposed symmetric key cryptographic system demonstrates promising capabilities in addressing information security concerns in long-distance transmissions, contributing to the advancement of network security.

Keywords: Information Security, Artificial Neural Networks, Feed Forward Neural Network, Symmetric Key Cryptography, Data Confidentiality, Message Authentication, Long-Distance Transmissions, Network Resilience.

Table of Contents

- Acknowledgement.....4
- Abstract.....5
- Introduction.....7
- Features of cryptography.....8
- Types of cryptography.....8
- Artificial Neural Network.....9
- Architecture of Neural Network..... 10
- Synapse Concept..... 12
- Concept of Axon Dendron..... 13
- Types of Neural Network..... 13
- Objective..... 16
- Tools Used..... 16
- Literature Survey..... 17
- Feasibility Study..... 18
- SRS..... 18
- Project Scheduling..... 19
- Flow diagram..... 20
- Algorithm..... 21
- Encryption..... 22
- Decryption..... 22
- Example Illustration..... 23
- Sample Output..... 27
- Applications.....28
- Testing..... 29
- Conclusion..... 30
- Future Scope..... 30
- Bibliography.....31
- Source Code..... 33

Introduction

The digital age has ushered in an era of unprecedented reliance on information. From personal health records to financial transactions, a vast amount of sensitive data resides on interconnected systems. This interconnectedness, however, exposes information to a constant barrage of cyber threats. These threats, perpetrated by malicious actors, can range from data breaches and unauthorized access to system infiltration and manipulation. The ever-evolving nature of these threats necessitates the development of robust and adaptable security solutions.

Artificial Neural Networks (ANNs) offer a promising approach to enhance information security. Inspired by the structure and learning capabilities of the human brain, ANNs are complex algorithms capable of identifying patterns, learning from vast amounts of data, and adapting to changing environments. This survey delves into the potential of ANNs to revolutionize information security practices. We will begin by exploring the fundamental concepts of cryptography, a cornerstone of information security, before examining the intricacies of neural networks and their potential synergy.

Features of Cryptography

Cryptography plays a critical role in safeguarding information. It utilizes complex mathematical algorithms to scramble data, rendering it unreadable by unauthorized parties. This process, known as encryption, ensures the confidentiality of information. Cryptography also guarantees the integrity of data by employing techniques that detect any unauthorized modifications. Additionally, it helps verify the authenticity of data, ensuring its origin and preventing impersonation attacks.

A symmetric cryptography has been implemented to ensure:

- **Confidentiality:** Ensures that only authorized users can access and understand the information. Encrypted data appears gibberish to anyone without the decryption key.
- **Integrity:** Guarantees that data remains unaltered during transmission or storage. Any unauthorized modification attempt will be flagged by the cryptographic system.
- **Authenticity:** Verifies the source of the information and prevents impersonation. Cryptographic techniques can confirm that the data originates from a trusted source and has not been tampered with en route.

Types of Cryptography

There are two main categories of cryptographic systems: symmetric and asymmetric.

- **Symmetric Cryptography:** This approach utilizes a single secret key for both encryption and decryption. Both parties

involved in secure communication must possess this shared key. Symmetric cryptography offers efficient encryption and decryption but requires secure key management, as a compromised key can expose all protected information.

- **Asymmetric Cryptography:** This method employs a key pair - a public key and a private key. The public key is widely distributed and used for encryption, while the private key is kept confidential and used for decryption. Asymmetric cryptography offers enhanced security for key management but can be computationally more expensive compared to symmetric methods.

Artificial Neural Network

Imagine a network of interconnected processing units mimicking the human brain's structure and learning capabilities. This is the essence of an Artificial Neural Network (ANN). These networks are not simply programmed with specific instructions; they learn from vast amounts of data, allowing them to identify patterns, make predictions, and adapt to changing circumstances.

ANNs consist of multiple layers of interconnected processing units called neurons. These neurons are inspired by biological neurons and are designed to receive, process, and transmit information. The connections between neurons, known as synapses, are analogous to the connections between biological neurons. The strength of these connections is influenced by

weights, which play a crucial role in the learning process of the network.

Here's a breakdown of the key capabilities of ANNs:

- **Pattern Recognition:** ANNs excel at identifying complex patterns in data. By analyzing large datasets, they can learn to recognize anomalies, suspicious activities, and potential security threats.
- **Learning:** ANNs are not limited to pre-programmed tasks. They can learn and improve their performance over time by being exposed to new data. This continuous learning ability is particularly valuable in the dynamic and ever-evolving landscape of information security.
- **Adaptation:** ANNs can adjust their internal structure and weights based on new information. This allows them to adapt to changing threats and security challenges.

The potential applications of ANNs in information security are vast and hold immense promise for creating a more secure digital environment.

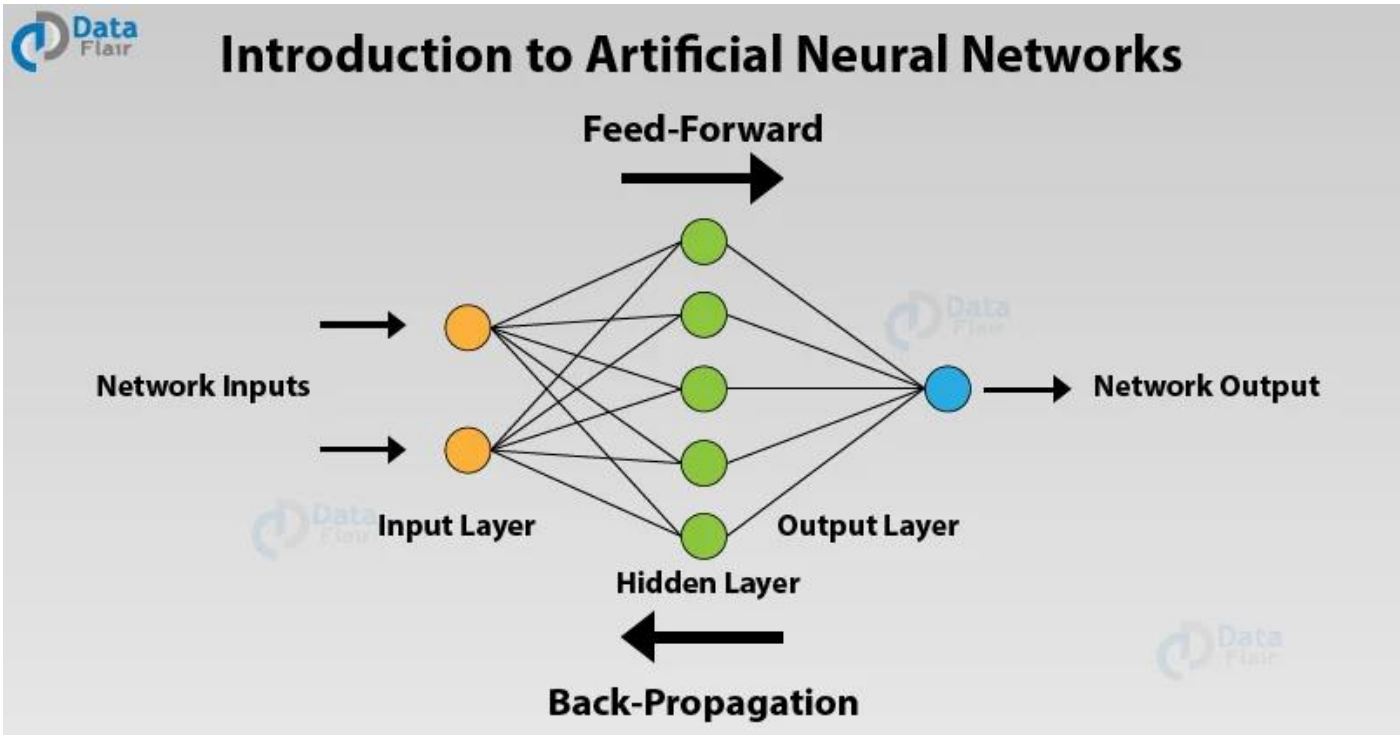
Architecture of Neural Network

An ANN is built upon a layered structure, much like the human brain. These layers are interconnected, allowing information to flow and be processed throughout the network. Here's a breakdown of the key components:

- **Neurons:** The fundamental processing units of an ANN. Each neuron receives input from other neurons, performs a

simple calculation, and transmits a signal to connected neurons.

- **Layers:** Neurons are organized into layers, with information flowing from the input layer to the output layer through hidden layers (if present).
    - **Input Layer:** The first layer receives raw data, which can be network traffic data, system logs, or any security-related information relevant to the specific application.
    - **Hidden Layers:** These layers, situated between the input and output layers, perform the core computations. They extract complex features from the data and enable the network to learn intricate relationships within the information. The number of hidden layers and the number of neurons within each layer can vary depending on the complexity of the task the network is designed to perform.
    - **Output Layer:** The final layer produces the network's output, which could be a classification (e.g., normal vs. malicious activity) or a prediction (e.g., likelihood of a security breach).



Figure(1): Architecture of ANN

Synapse Concept

Neurons communicate with each other through connections called synapses. These synapses act like bridges, transmitting signals between neurons. The strength of the signal transmitted is influenced by a weight associated with each synapse. Imagine weights as volume knobs - a higher weight allows a stronger signal to pass through, influencing the receiving neuron more significantly.

During the training process, these weights are adjusted based on the network's performance. If the output is incorrect, the weights are fine-tuned to improve the accuracy of future predictions. This process allows the network to learn and adapt over time.

Concept of Axon and Dendron

Within a single neuron, information flows in a specific direction. Dendrites, which resemble branching structures, act as receivers, gathering input signals from other neurons through connected synapses. The collected signals are then processed by the neuron's internal mechanism. The processed information is then transmitted through a single axon, another branching structure, to connected neurons via their synapses.

The interplay between neurons, synapses, axons, and dendrites allows ANNs to process information in a complex and distributed manner, enabling them to learn from data and make informed decisions.

Types of Neural Network Architectures

There are various ANN architectures, each suited to specific tasks and data types. Here are a few common examples:

- **Perceptrons:** These are the simplest form of ANNs, consisting of a single input layer, a single hidden layer with one neuron, and an output layer. While limited in their capabilities, they serve as the building block for more complex architectures.
- **Multilayer Perceptrons (MLPs):** These are ANNs with multiple hidden layers, allowing them to learn more intricate relationships within the data. MLPs are widely used for various tasks, including pattern recognition and classification, making them well-suited for information security applications.

- **Convolutional Neural Networks (CNNs):** These architectures excel at processing data with a grid-like structure, such as images and time series data. CNNs are particularly effective in applications involving image recognition and anomaly detection in network traffic data.
- **A feedforward neural network (FNN)**: These is a fundamental type of artificial neural network where the connections between the units (neurons) do not form cycles. In other words, the information moves in one direction, forward, from the input nodes through the hidden layers (if present) to the output nodes. It's one of the simplest types of neural networks but can be incredibly powerful for various tasks, including classification, regression, and pattern recognition.

These are just a few examples, and new ANN architectures are constantly being developed to address various challenges. The choice of architecture depends on the specific information security task at hand and the nature of the data being analyzed.

By understanding the different components and functionalities of ANNs, we can begin to explore their potential applications in safeguarding information security.

The Intersection of ANNs and Information Security

Having explored the fundamentals of ANNs, we can now delve into their potential applications within the realm of information security. Here are some promising areas where ANNs can be leveraged to bolster our defenses:

- **Malware Detection:** ANNs can be trained on vast datasets of malware and benign software samples. This training enables them to identify malicious code with high accuracy based on patterns and characteristics present in the code. This can be crucial in proactively detecting and preventing malware infections.
- **Spam Filtering:** Spam emails can be a nuisance and a security threat. ANNs can analyze email content, sender information, and other email features to distinguish between legitimate emails and spam. This can significantly reduce the volume of spam reaching user inboxes and improve overall email security.
- **Anomaly Detection:** ANNs excel at identifying deviations from normal patterns. This capability can be harnessed to detect anomalous activity within systems, such as unauthorized access attempts or unusual resource utilization. By flagging such anomalies, security teams can investigate potential threats and take preventive measures.
- **Data Security:** ANNs can be used to secure sensitive data by implementing techniques like anomaly detection in data access patterns or building models to predict and prevent unauthorized data exfiltration attempts.

Objective is to develop a novel symmetric key cryptography system using feed forward neural architecture to ensure data confidentiality and message authentication during long-distance transmissions across computer networks**.** This system aims to achieve the following:

Enhanced Data Confidentiality: Utilize the power of ANNs to create a dynamic and unpredictable encryption process, making it significantly more difficult for unauthorized parties to decipher the transmitted data.

Robust Message Authentication: Incorporate ANNs to establish a secure method for verifying the message's origin and preventing tampering during transmission. This ensures data integrity and prevents forgery attempts.

Long-Distance Network Resilience: Design the system specifically for long-distance transmissions across diverse computer networks. This includes addressing potential network delays and ensuring the system remains effective even with varying data packet sizes.

TOOLS USED FOR COMPLETION

- **Programming language: Python**
- **Operating System: Windows 11**
- **Statistical project view app: MS Project**
- **Writeup app: MS Word**

Literature Survey :

[1]Marek Pawlicki, Rafał Kozik, Michał Choras focuses the challenges of implementing Artificial Neural Networks (ANNs) for intrusion detection systems (IDS) in cybersecurity. It combines a review of existing literature with the authors' practical experience to explore key issues and potential solutions.

[2] Kushal Karmakar, Soumik Kundu, Soumya Paul describes data transmission using symmetric key cryptography with the help of neural network architecture supporting Multilayer Feed forward Network where a random key is generated & it is further coded with the hidden layers to make it more secured .

[3] Soumya Paul et.al [1] concerns about the security of confidential information and data transmission using public key cryptography with Genetic Algorithm in order to provide confidentiality, authentication, integrity and non-repudiation of the messages. [

[4] Tragha, F. Omary, A. Mouloudi, explores the potential of deep learning (DL) for improving cybersecurity in IoT networks and beyond.

[5] Subhrajit Mondal, Tania Khatun Mollah, Arindam Samanta, and Soumya Paul state this about the security of confidential information and data transmission using symmetric key cryptography with Genetic Algorithm in order to provide confidentiality, authentication, integrity and non-repudiation of the messages.

[6] JONGHOON LEE , JONGHYUN KIM , IKKYUN KIM ,ANDKIJUN HAN , aims to assess the effectiveness of an AI technique for cyber-threat detection using artificial neural networks.

[7] Ece Cantemir, Ozlem Kandemir investigates the applications of artificial intelligence (AI) in architecture, with a specific focus on convolutional neural networks (CNNs) for architectural style classification.

Feasibility Study

Technical Feasibility: This study involves the assessment of the proper skill strength of the organization. It assesses whether the project is feasible in terms of engineering complexity.

Behavioural Feasibility: It evaluates and estimates the user attitude or behaviour towards the development of new system. It helps in determining if the system requires special effort to educate, retrain, transfer, and changes in employee's job status on new ways of conducting business.

Software Requirement Specification (SRS)

Functional Requirement: The proposed information security system comprises an encryption module, decryption module, and key generation module to ensure the confidentiality and authentication of messages. The encryption module utilizes neural network algorithms to conceal information effectively, while the decryption module is responsible for decoding the encrypted data at the recipient's end.

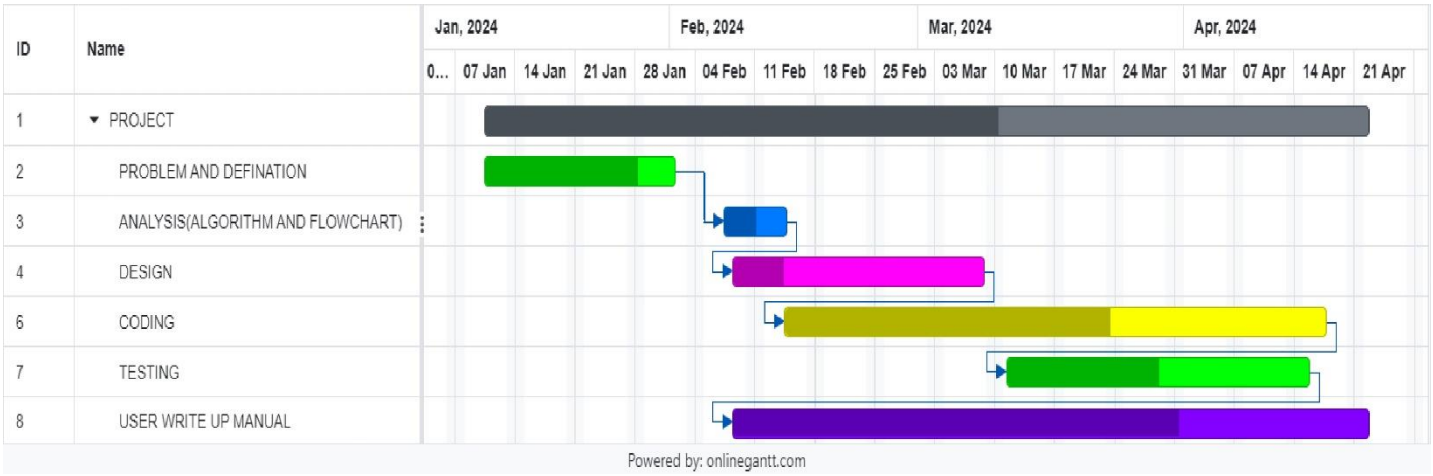
Non-Functional Requirement: This system must adhere to stringent security standards to safeguard sensitive information. It should demonstrate reliability in encrypting and decrypting data, ensuring that unauthorized parties cannot access the information. Additionally, the system should exhibit high performance, maintaining efficiency even under heavy loads. It must be easily maintainable to address any potential vulnerabilities promptly. Furthermore, scalability is crucial to accommodate growing data volumes and user demands. Finally, usability is essential to ensure that the system is intuitive and accessible to authorized users.

Goals of Implementation: The primary objective of implementing this algorithm in information security is to guarantee the confidentiality and authenticity of messages exchanged within computer systems. By leveraging advanced encryption techniques and robust key management, the system aims to prevent unauthorized access to sensitive data and protect against malicious activities such as eavesdropping and tampering.

Project Scheduling:

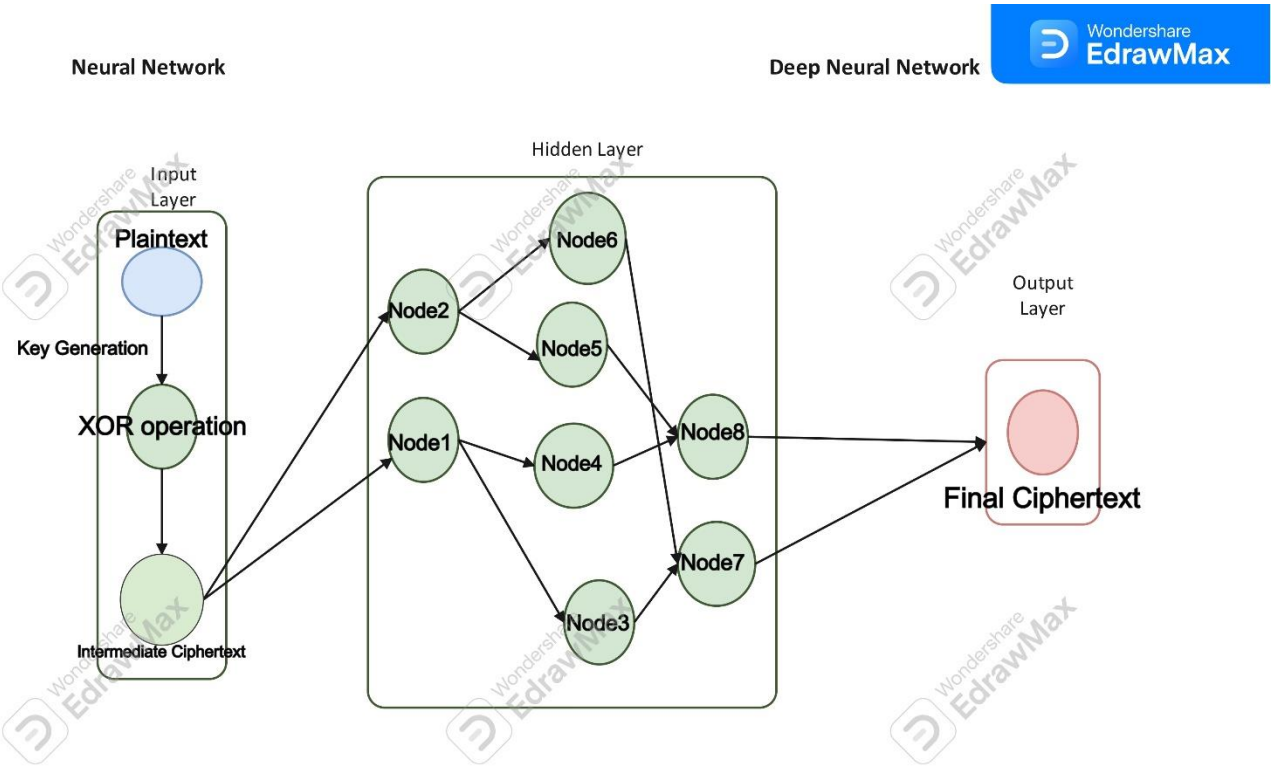
TASK	START DATE	END DATE	Duration in days
PROBLEM DEFINATION AND LITERATURE SURVEY	10-01-2024	01-02-2024	22
ANALYSIS(ALGORITHM AND FLOWCHART)	07-02-2024	14-02-2024	7
DESIGN	08-02-2024	08-03-2024	29
CODING	14-02-2024	17-04-2024	63
TSESTING	11-03-2024	15-04-2024	36
USER WRITE UP MANUAL	08-02-2024	20-04-2024	72

Figure(2): Gantt Table



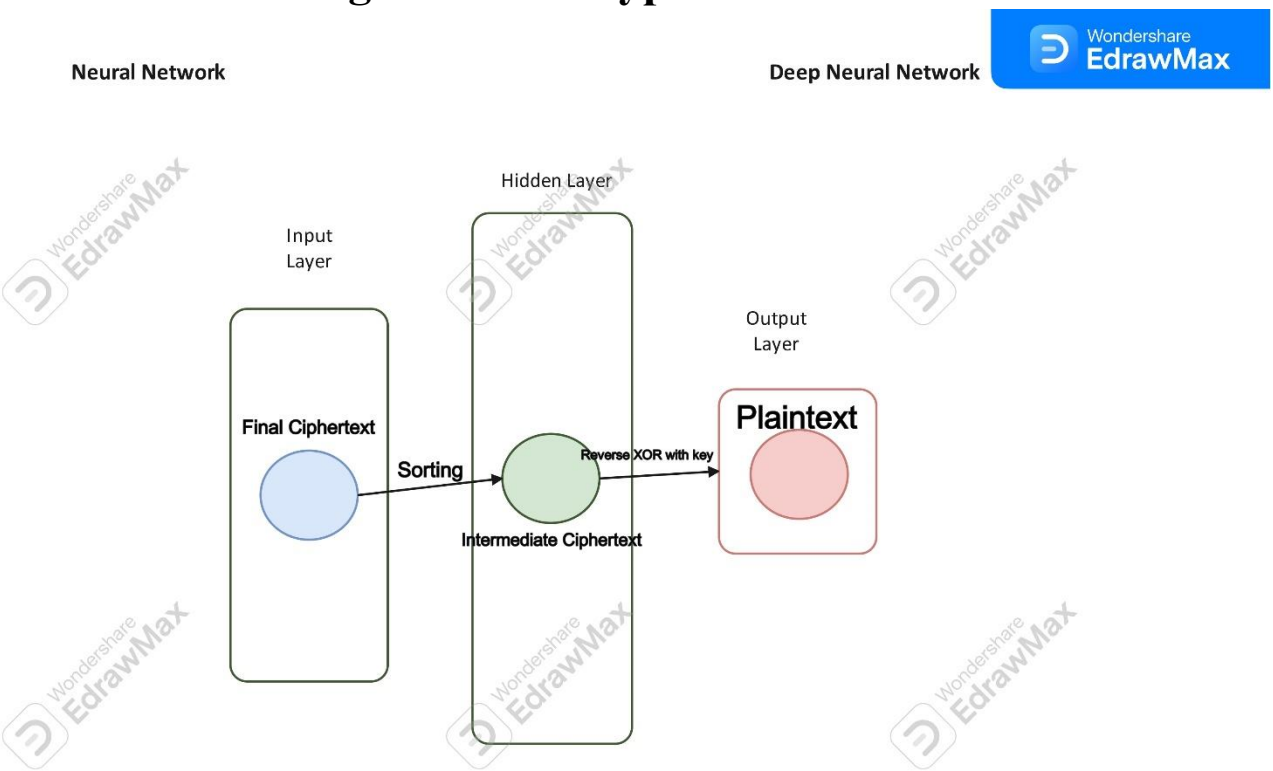
Figure(3): Gantt Chart

Flow Diagram of Encryption Heuristic:



Figure(4): Encryption Heuristic Flow Diagram

Flow Diagram of Decryption Heuristic:



Figure(5):Decryption Heuristic Flow Diagram

Algorithms

Main Algorithm

Step 1: Take the plain text string as input.

Step 2: Call the key generation algorithm.

Step 3: Call the encryption algorithm.

Step 4: Cipher text generated.

Step 5: Call the decryption algorithm.

Step 6: Plain text retrieved.

Step 7: Stop.

Key Generation Heuristic:

1. Take the plaintext as input.
2. Determine the key length.
3. Repeat the plaintext to match the key length or exceed it.
4. Calculate the ASCII values of each character in the key part.
5. Generate the key by taking the absolute difference between consecutive ASCII values.
6. Convert the ASCII values into characters
7. Return the generated key.

8. Stop.

Encryption Heuristic:

1. Convert the plaintext into ASCII values.
2. Convert the key into ASCII values.
3. Iterate through each character of the plaintext and key.
4. Perform XOR operation between corresponding ASCII values of the plaintext and key.
5. Convert the result of XOR operation back to characters.
6. For every non-printable character, replace it with ^ and store the index of each.
7. Store the result of each XOR operation.
8. Return the intermediate encrypted text 1.
9. Store the index of each character and form a list with the characters and their index.
10. Split the encrypted characters into alphabetic and non-alphabetic parts as Node1 and Node2 respectively.
11. Split Node1 and Node2 based on Even and Odd Indices- Node 3 (Even Index Alphabetic Characters),Node 4 (Odd Index Alphabetic Characters),Node 5 (Even Index Non-Alphabetic Characters),Node 6 (Odd Index Non-Alphabetic Characters)
12. Concatenate the Nodes in this order, Node3,Node6,Node4,Node5.
13. Return the final cipher.
14. Stop.

Decryption Heuristic:

1. Sort the combined characters based on their indices.

2. Decode any encoded null characters back to null characters.
3. Do reverse XOR operation on each character with the key.
4. Return the decrypted plaintext.
5. Stop.

Example Illustrating Key Generation:

Step 1: - Insert a plain text.

- Namaste India!

Step 2: - Compute the length of the string.

- Length of the string =14

Step 3: - If the length of the string is less than or equal to 8 then the length of the string else the first 8 characters of the string is taken.

- Namaste

Step 4: - Convert the 8-bit string into its corresponding ASCII value.

"N": 78
"a": 97
"m": 109
"a": 97
"s": 115
"t": 116
"e": 101
" ": 32 (space)

Step 5: - Compute the difference between two consecutive ASCII values and perform the modulus function on them. [The last bit is subtracted from the first bit and modulus function is performed.]

('N', 'a'): ASCII values (78, 97), Difference = 78 - 97 = -19, Modulus = (-19) % 95 = 76
('a', 'm'): ASCII values (97, 109), Difference = 97 - 109 = -12, Modulus = (-12) % 95 = 83
('m', 'a'): ASCII values (109, 97), Difference = 109 - 97 = 12, Modulus = 12 % 95 = 12
('a', 's'): ASCII values (97, 115), Difference = 97 - 115 = -18, Modulus = (-18) % 95 = 77
('s', 't'): ASCII values (115, 116), Difference = 115 - 116 = -1, Modulus = (-1) % 95 = 94

('t', 'e'): ASCII values (116, 101), Difference = 116 - 101 = 15, Modulus = 15 % 95 = 15
('e', ' '): ASCII values (101, 32), Difference = 101 - 32 = 69, Modulus = 69 % 95 = 69
(' ', 'N'): ASCII values (32, 78), Difference = 32 - 78 = -46, Modulus = (-46) % 95 = 49

Step 7: - The ASCII value is taken and converted to its corresponding characters.
('N', 'a'): ASCII values (78, 97), Difference = -19, Modulus = 76, Character: 'L'
('a', 'm'): ASCII values (97, 109), Difference = -12, Modulus = 83, Character: 'S'
('m', 'a'): ASCII values (109, 97), Difference = 12, Modulus = 12, Character: '\x0C'
('a', 's'): ASCII values (97, 115), Difference = -18, Modulus = 77, Character: 'M'
('s', 't'): ASCII values (115, 116), Difference = -1, Modulus = 94, Character: '\'
('t', 'e'): ASCII values (116, 101), Difference = 15, Modulus = 15, Character: '\x0F'
('e', ' '): ASCII values (101, 32), Difference = 69, Modulus = 69, Character: 'E'
(' ', 'N'): ASCII values (32, 78), Difference = -46, Modulus = 49, Character: '1'

Step 7: - The string generated is our key.

- LS\x0CM\\x0FE1

Example illustrating Encryption:

Doing XOR operation on the corresponding ASCII values of the plain text and Key:

1. N (78) XOR L (76) = 78 ^ 76 = 2
2. a (97) XOR S (83) = 97 ^ 83 = 14
3. m (109) XOR \x0C (12) = 109 ^ 12 = 101
4. a (97) XOR M (77) = 97 ^ 77 = 48
5. s (115) XOR \ (92) = 115 ^ 92 = 39
6. t (116) XOR \x0F (15) = 116 ^ 15 = 103

7. e (101) XOR E (69) = 101 ^ 69 = 32
8. (32) XOR 1 (49) = 32 ^ 49 = 17
9. I (73) XOR L (76) = 73 ^ 76 = 5
10. n (110) XOR S (83) = 110 ^ 83 = 61
11. d (100) XOR \x0C (12) = 100 ^ 12 = 108
12. i (105) XOR M (77) = 105 ^ 77 = 32
13. a (97) XOR \ (92) = 97 ^ 92 = 29
14. ! (33) XOR \x0F (15) = 33 ^ 15 = 46

Intermediate Encrypted Text:
\x02\x0Ee0'g \x11\x05=l \x1d

Store the indices of each character in a list:
[('\x02', 0), ('\x0E', 1), ('e', 2), ('0', 3), ('"', 4), ('g', 5), (' ', 6), ('\x11', 7), ('\x05', 8), ('=', 9), ('l', 10), (' ', 11), ('\x1d', 12), ('.', 13)]

Now divide the ciphertext into Node1 and Node2 consisting of Alphabetic and non alphabetic characters respectively:

Node1 (Alphabetic characters): [('e', 2), ('g', 5), ('l', 10)]
Node2 (Non-alphabetic characters): [('\x02', 0), ('\x0E', 1), ('0', 3), ('"', 4), (' ', 6), ('\x11', 7), ('\x05', 8), ('=', 9), (' ', 11), ('\x1d', 12), ('.', 13)]

Now further split the alphabetic characters into Node3 and Node4 according to even and odd indices respectively. do the same for non alphabetic characters into Node5 and Node6:

Node3 (Alphabetic characters - Even indices): [('e', 2)]

Node4 (Alphabetic characters - Odd indices): [('g', 5), ('l', 10)]

Node5 (Non-alphabetic characters - Even indices): [('\x02', 0), ('0', 3), (' ', 6), ('\x05', 8), ('=', 9), ('\x1d', 12)]

Node6 (Non-alphabetic characters - Odd indices): [('\x0E', 1), ('"', 4), ('\x11', 7), (' ', 11), ('.', 13)]

Now concatenate node 3 and 6 and Node 4 and 5 to form nodes 7 and 8 respectively:

Node7 (Concatenated from Node3 and Node6): [('e', 2), ('\x0E', 1), ("'", 4), ('\x11', 7), (' ', 11), ('.', 13)]

Node8 (Concatenated from Node4 and Node5): [('g', 5), ('l', 10), ('\x02', 0), ('0', 3), (' ', 6), ('\x05', 8), ('=', 9), ('\x1d', 12)]

Now concatenate node 7 and 8 to form the final cipher:

Final Cipher: [('e', 2), ('\x0E', 1), ("'", 4), ('\x11', 7), (' ', 11), ('.', 13), ('g', 5), ('l', 10), ('\x02', 0), ('0', 3), (' ', 6), ('\x05', 8), ('=', 9), ('\x1d', 12)]

Example illustrating Decryption:
We have stored the indices of the characters before splitting them into nodes.

Sort the indices:

[('\x02', 0), ('\x0E', 1), ('e', 2), ('0', 3), ("'", 4), ('g', 5), (' ', 6), ('\x11', 7), ('\x05', 8), ('=', 9), ('l', 10), (' ', 11), ('\x1d', 12), ('.', 13)]

Now do reverse XOR operation with the key:

It will give Namaste India!

Output:

```
Enter the plaintext to encrypt:
Hello Everyone! Welcome to our project.
Generated Key: .  O%1
Intermediate encrypted text: fxkllo`GKo~om*  yxkcl"@ Zr'ov= A\rme`;

Final Ciphertext: fklGomxcZoAm~"=\;xloKoyklrvre`*@'`
Intermediate ciphertext: fxkllo`GKo~om*  yxkcl"@ Zr'ov= A\rme`;

Decrypted: Hello Everyone! Welcome to our project.


** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

```
Enter the plaintext to encrypt:
May everyone stay happy.
Generated Key: %  YE
Intermediate encrypted text: huay gt \{v<ebel\4p85ah#
Final Ciphertext: hagvbla{\8#uyteeph \<45
Intermediate ciphertext: huay gt \{v<ebel\4p85ah#
Decrypted: May everyone stay happy.


** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

```
Enter the plaintext to encrypt:
Can you please look into it? It's very urgent!
Generated Key: #
NY
U
Intermediate encrypted text: `cn esuSrh/*o&9Lqfn0dr:wyqyCrrP>{++s& Qyh -+
Final Ciphertext: ceuroqnryyrPQh/&0{+ -nsShLfdwqCrsy` *9:>+& +
Intermediate ciphertext: `cn esuSrh/*o&9Lqfn0dr:wyqyCrrP>{++s& Qyh -+
Decrypted: Can you please look into it? It's very urgent!


** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Applications:

The project investigating the application of artificial neural networks (ANNs) in enhancing information security can have several practical applications across various domains. Here are some potential applications:

Network Security: Implementing the ANN model for intrusion detection systems (IDS) in network security. ANNs can analyze network traffic patterns in real-time to identify and mitigate potential threats such as malware, DDoS attacks, and unauthorized access attempts.

Cybersecurity for Financial Institutions: Deploying ANNs to detect fraudulent activities and enhance cybersecurity measures in financial institutions. ANNs can analyze transactional data to identify anomalous patterns indicative of fraudulent behavior, helping to prevent financial losses and protect customer assets.

Data Protection in Healthcare: Utilizing ANNs to enhance data protection and privacy in healthcare systems. ANNs can be employed to encrypt sensitive patient information, ensuring compliance with regulatory standards such as HIPAA while safeguarding patient confidentiality.

E-commerce Security: Integrating ANNs into e-commerce platforms to enhance security measures for online transactions. ANNs can analyze user behavior, transaction history, and payment data to detect and prevent fraudulent activities such as credit card fraud and identity theft.

Endpoint Security: Employing ANNs for endpoint security solutions to protect devices and endpoints from malware, ransomware, and other cyber threats. ANNs can analyze system behavior and identify suspicious activities, enabling proactive threat detection and response.

Critical Infrastructure Protection: Implementing ANNs for securing critical infrastructure such as power plants, transportation systems, and water treatment facilities. ANNs can analyze operational data to detect cyber threats and potential vulnerabilities, ensuring the resilience and reliability of critical infrastructure systems.

Cloud Security: Leveraging ANNs to enhance security in cloud computing environments. ANNs can monitor network traffic, detect unauthorized access attempts, and provide anomaly detection capabilities to mitigate security risks in cloud-based applications and services.

Mobile Security: Integrating ANNs into mobile security solutions to protect smartphones and mobile devices from malware, phishing attacks, and other cyber threats. ANNs can analyze app behavior, network communications, and device activity to identify and mitigate security risks.

Testing :

Thorough testing is crucial before releasing software to clients:

White box testing: It offers a security advantage for neural networks by leveraging their internal structure to design targeted tests for vulnerabilities, uncover hidden biases, and improve the

overall security of these increasingly used information security systems.
Unit Testing: Individual modules are tested in isolation to ensure they function correctly according to their specifications.
Integration Testing: All the modules are combined and tested as a whole system to verify they interact seamlessly and deliver the expected overall functionality.
This combined approach helps identify and fix issues early in development, leading to a more robust and reliable software product.

Conclusion :

In conclusion, this project investigated the application of artificial neural networks (ANNs) in enhancing information security. The implemented ANN model demonstrated its potential to effectively learn and detect security threats from data. In addition to its effectiveness in threat detection, the utilization of artificial neural networks showcased significant advancements in data encryption techniques. By employing neural networks, encryption processes can be enhanced, offering stronger security measures against unauthorized access and data breaches.

Future scope :

* Asymmetric cryptography maybe implemented to provide a foundation for secure communication between different parties involved in the computation.

* Complex mathematical functions could be incorporated into the neural network architecture to enhance its processing capabilities for specific tasks.

* Dynamic node generation would allow the neural network to adapt its structure during computation, potentially improving efficiency and accuracy.

Overall, this future scope presents an exciting opportunity to explore the intersection of cryptography, complex mathematics, and neural networks to create novel solutions for secure multi-party computation.

Bibliography :

[1] Marek Pawlicki, Rafał Kozik, Michał Choras, “A survey on neural networks for (cyber-) security and (cyber-) security of neural networks”, ELSEVIER, Volume 500, 21 August 2022, Page 1075-87

[2] Kushal Karmakar, Soumik Kundu, Soumya Paul, “Design and Implementation of Information Security using Neural Network Architecture”, International Journal of Applied Engineering Research ISSN 0973-4562 Volume 13, Number 11 (2018) pp. 9916-9923

[3] Soumya Paul et.al, Implementation of Network Security Using Genetic Algorithm, Vol 3 Issue 2, February-2013 ISSN:2277-128X A.

[4] A. Tragha, F. Omary, A. Mouloudi, “Improved Cryptography Inspired by Genetic Algorithms”, ICIGA, 2006 International Conference on Hybrid Information Technology (ICHIT'06).

[5] Subhrajit Mondal, Tania Khatun Mollah, Arindam Samanta, Soumya Paul, A Survey on Network Security Using Genetic Algorithm, International Journal of Innovative Research in Science, Engineering and Technology (A High Impact Factor, Monthly Peer Reviewed Journal) Vol. 5, Issue 1, January 2016.

[6] JONGHOON LEE , JONGHYUN KIM , IKKYUN KIM ,ANDKIJUN HAN, “Cyber Threat Detection Based on Artificial Neural Networks Using Event Profiles”, IEEE Access , Volume 7,2019

[7] Ece Cantemir, Ozlem Kandemir, “Use of artificial neural networks in architecture: determining the architectural style of a building with a convolutional neural networks”, Springer, Volume 36,2024, page 6195-6207

[8] William Stallings, Cryptography and Network Security, 2nd edition.

[9] Neural Network: An approach for Cryptography by Puneet Goswami.

[10] Cryptography & Network Security by Atul Kahate.

[11] Fundamentals of Software Engineering by Rajib Mall.

[12] Software Engineering, a practitioner's approach by Roger S. Pressman.

Source Code:

```
def generate_key(plaintext):
    """Generates a key based on the plaintext."""
    key_length = 8
    key_part = (plaintext * (key_length // len(plaintext) + 1))[:key_length]
    ascii_values = [ord(char) for char in key_part]
    key = ''.join(chr(abs(ascii_values[i] - ascii_values[i-1])) for i in
range(key_length))
    return key

def encode_null_chars(plaintext):
    """Encode null characters in the plaintext."""
    encoded_text = plaintext.replace('\x00', '<NULL>')  # Example
encoding for null characters
    return encoded_text

def decode_null_chars(encoded_text):
    """Decode the encoded null characters back to null characters."""
    plaintext = encoded_text.replace('<NULL>', '\x00')  # Example
decoding for null characters
    return plaintext
```

```
def encrypt(plaintext, key):


    plaintext = encode_null_chars(plaintext)
    """Encrypts plaintext, handling non-printable characters and additional steps."""
    skipped_indices = []
    skipped_chars = []
    encrypted_chars = []
    combined_chars = []
    intermediate_encrypted_text = ""

    for i, char in enumerate(plaintext):
        key_char = key[i % len(key)]  # Cycle through key
        encrypted_char = chr(ord(char) ^ ord(key_char))
        intermediate_encrypted_text += encrypted_char


        if not encrypted_char.isprintable():
            skipped_indices.append(i)
            skipped_chars.append(char)
            encrypted_char = '\x00'  # Use the null character as a placeholder for non-printable characters
```

```
        encrypted_chars.append(encrypted_char)
        combined_chars.append((encrypted_char, i))

    # Sort combined characters and indices
    combined_chars.sort(key=lambda x: x[1])

    # Split and concatenate
    alpha_chars = ''.join(c for c in encrypted_chars if c.isalpha())
    non_alpha_chars = ''.join(c for c in encrypted_chars if not c.isalpha())
    alpha_even = alpha_chars[::2]
    alpha_odd = alpha_chars[1::2]
    non_alpha_even = non_alpha_chars[::2]
    non_alpha_odd = non_alpha_chars[1::2]
    encrypted_text = ''.join([alpha_even, non_alpha_odd, alpha_odd,
non_alpha_even])

    return encrypted_text, skipped_indices, skipped_chars,
combined_chars, intermediate_encrypted_text

def decrypt(ciphertext, key, skipped_indices, skipped_chars,
combined_chars, intermediate_encrypted_text):
    """Decrypts the encrypted text using the provided key and combined
characters."""
```

```
    print("Intermediate ciphertext:", intermediate_encrypted_text)

    sorted_combined_chars = sorted(combined_chars, key=lambda x:
x[1])

    plaintext_chars = []
    for char, index in sorted_combined_chars:
        if index in skipped_indices:

plaintext_chars.append(skipped_chars[skipped_indices.index(index)])
# Replace the null character with the original character
        else:
            plaintext_char = chr(ord(char) ^ ord(key[index % len(key)]))
            plaintext_chars.append(plaintext_char)



    plaintext = ''.join(plaintext_chars)
    plaintext = decode_null_chars(plaintext)
    return plaintext

import sys

# Function to take user input as plaintext
def take_user_input():
```

```
    try:
        plaintext = input("Enter the plaintext to encrypt: ")
        if not plaintext:
            raise ValueError("Please enter some text.")
        return plaintext
    except KeyboardInterrupt:
        print("\nPrograminterrupted by the user. Exiting...")
        sys.exit()
    except Exception as e:
        print("An error occurred:", e)
        sys.exit()

# Example usage with user input
plaintext = take_user_input()  # Prompt user to enter plaintext
key = generate_key(plaintext)
print("Generated Key:", key)  # Print the generated key
ciphertext, skipped_indices, skipped_chars, combined_chars,
intermediate_encrypted_text = encrypt(plaintext, key)
print("Intermediate encrypted text:", intermediate_encrypted_text)
print("Final Ciphertext:", ciphertext)
```

```
decrypted_text = decrypt(ciphertext, key,
skipped_indices,skipped_chars, combined_chars,
intermediate_encrypted_text)

print("Decrypted:", decrypted_text)
```